



Faculty of Computer Science

Data Science

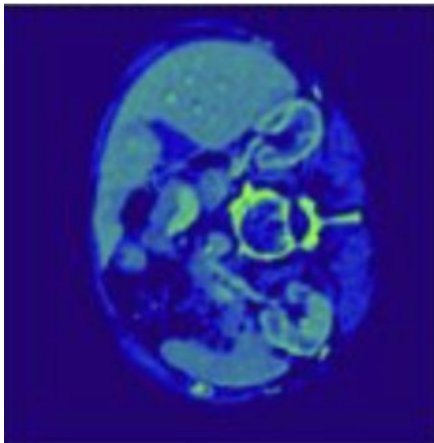
Moscow 2024

U-Net: Convolutional Networks for Biomedical Image Segmentation Review

Alina Prolygina

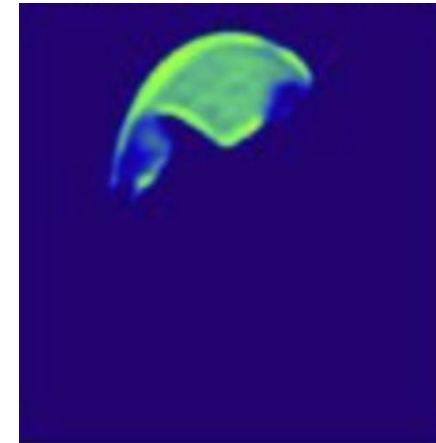
Problems

- Challenges in Biomedical Image Segmentation:
 - The need for precise pixel-wise classification.
 - Limited availability of annotated training data.
 - High complexity of biomedical images (overlapping structures, irregular shapes, or poor contrast).
- Examples of Tasks:
 - Neuron segmentation in electron microscopy.
 - Cell segmentation in light microscopy.



Solution

- U-Net architecture:
 - Symmetrical U-shaped structure.
 - Efficient combination of context and localization.
 - High accuracy with minimal data requirements.
- Results:
 - Leading performance in ISBI challenges on electron and light microscopy image segmentation.
 - Applicability to a wide range of segmentation tasks.



Role of Deep CNN in Image Processing

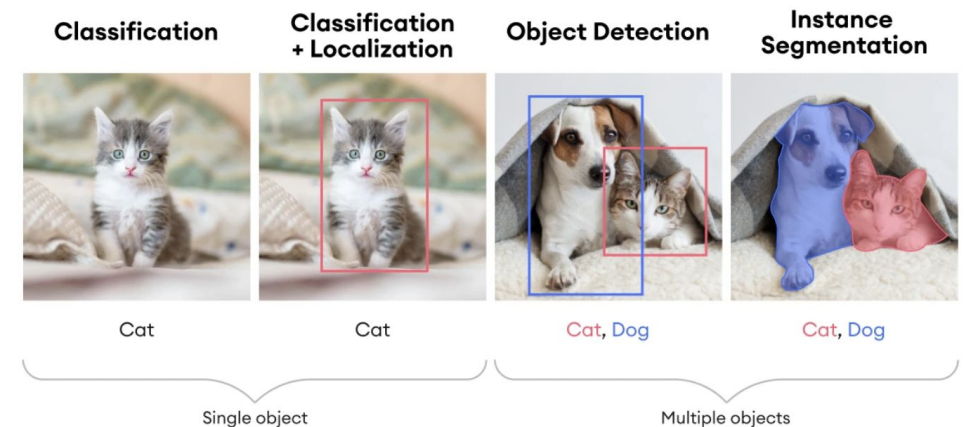
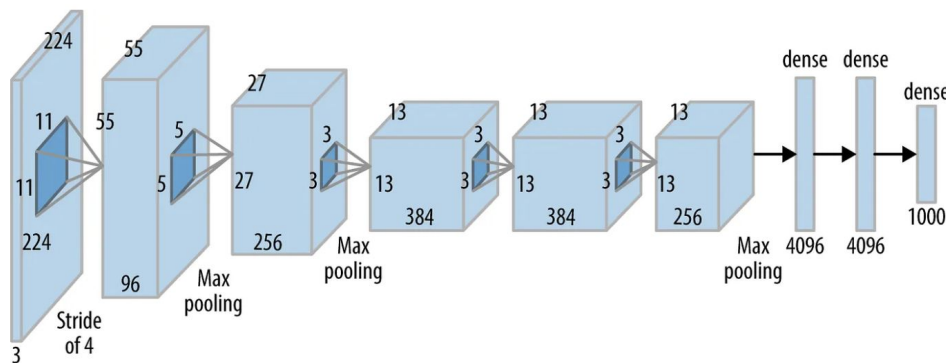
- Enable automatic feature extraction.
- Set new standards of accuracy in classification and recognition tasks.

Contribution of Krizhevsky et al. (2012)

- AlexNet architecture achieved outstanding results on ImageNet.
- Utilized deep convolutional networks trained on a dataset of 1 million images.

Shift from Classification to Segmentation

- Classification: assigning a single class label to an image.
- Segmentation: assigning a class label to each pixel.
- New challenges: ensuring localization accuracy and leveraging contextual information.

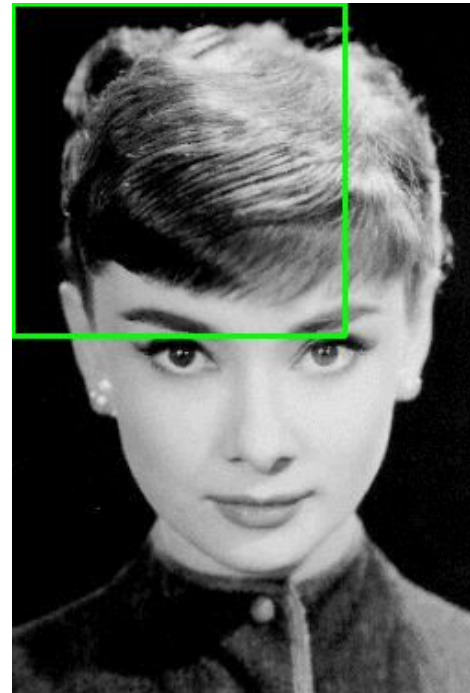


Sliding window method

- Applies a small window to all parts of the image.
- Each window is classified independently.

Drawbacks

- High computational cost: processing overlapping regions increases resource usage.
- Limited context: small windows only capture local features.
- Trade-off: larger windows provide more context but reduce localization accuracy.

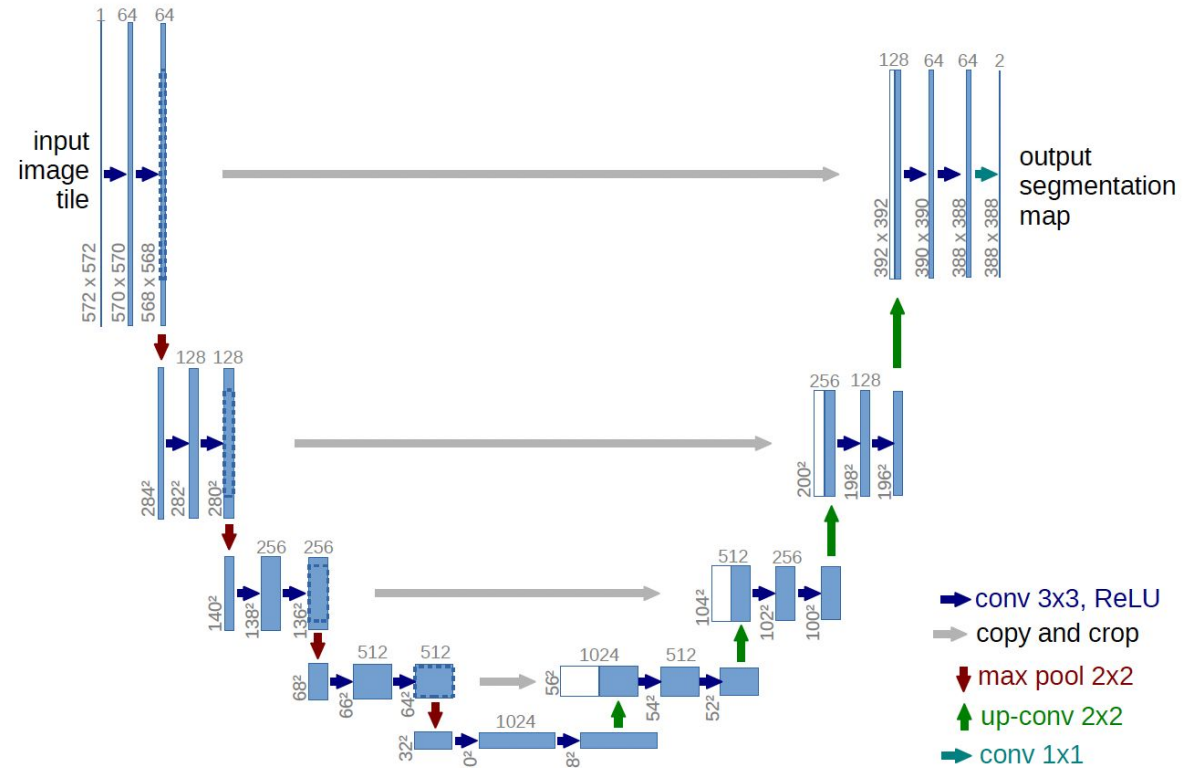


Advantages

- Simple implementation.
- Ability to localize objects.

U-Net Architecture

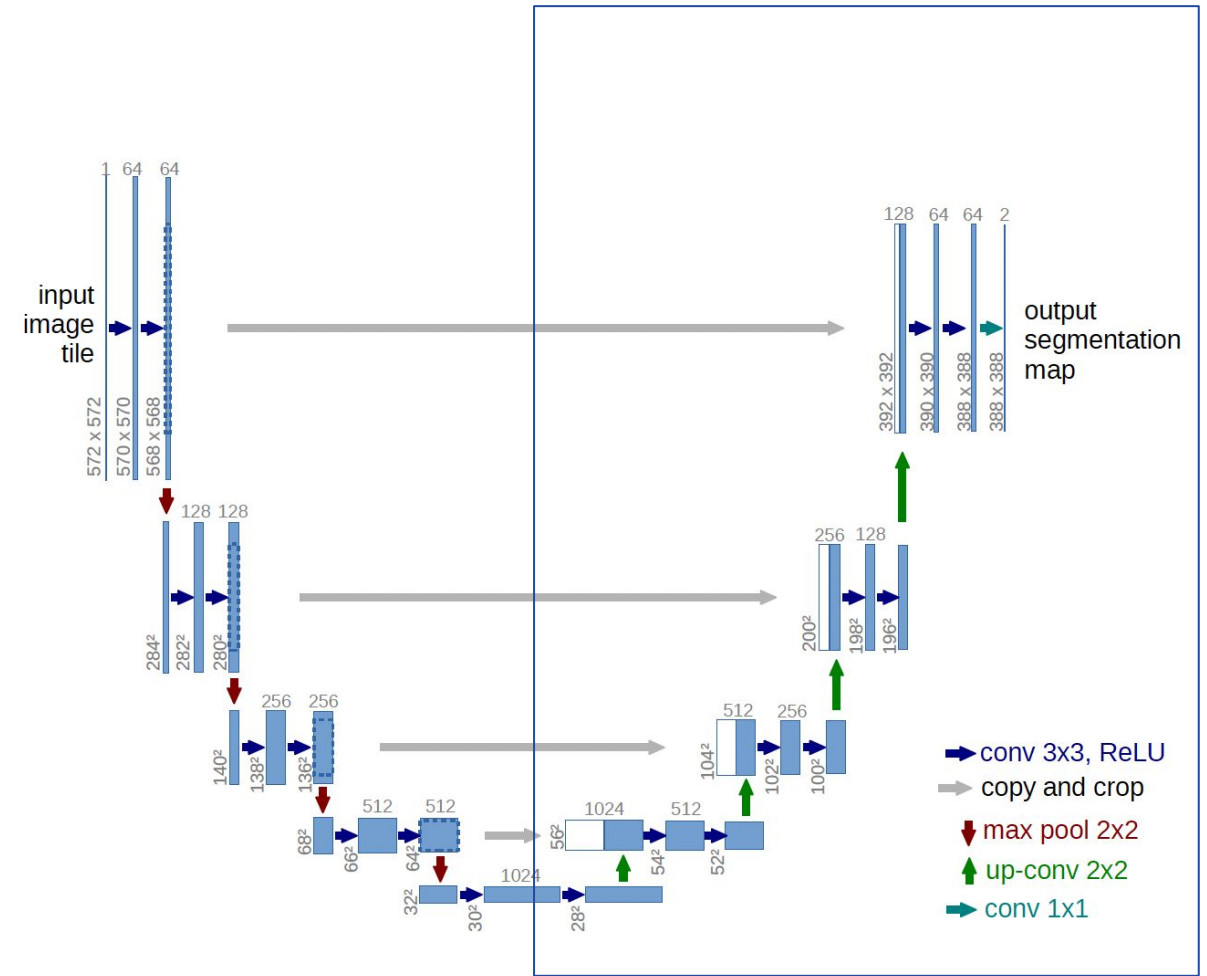
- Core Concept:
 - U-Net is designed specifically for pixel-wise image segmentation.
 - The architecture combines two main paths:
 - Contracting path: captures global context by progressively reducing spatial dimensions.
 - Expanding path: restores resolution for precise localization while incorporating contextual information.
- Key Characteristics:
 - Symmetrical, U-shaped structure.
 - Skip connections between corresponding layers in contracting and expanding paths.
 - Allows the network to integrate low-level spatial features with high-level semantic information.
 - Efficient for tasks with limited annotated data.



-
- The diagram illustrates the U-Net architecture for medical image segmentation. The input is a 512 x 512 x 3 image tile. The encoder (left) consists of four stages of 3x3 convolutions and max pooling. The bottleneck (middle) is a single 3x3 convolution. The decoder (right) consists of four stages of up-convolutions and 3x3 convolutions. Skip connections (top) are used to fuse features from the encoder to the decoder. The output is a segmentation map of size 512 x 512 x 2.
- Legend:**
- Blue arrow: conv 3x3, ReLU
 - Grey arrow: copy and crop
 - Red arrow: max pool 2x2
 - Green arrow: up-conv 2x2
 - Teal arrow: conv 1x1

Expanding Path

- Process Description:
 - Gradual restoration of spatial resolution.
 - The following operations are applied:
 - Up-Convolution (2x2): Increases the size of feature maps.
 - Merging with skip connections: Combines high-level features with detailed data from the contracting path.
 - 3x3 Convolutions and ReLU: Generates new feature maps with improved localization.
- Role of the Expanding Path:
 - Restores image resolution for precise segmentation.
 - Integrates contextual data with local features.
- Key Features:
 - Symmetry with the contracting path.
 - Use of skip connections for enhanced segmentation accuracy.



Key Modifications in U-Net

Core Innovations:

- Skip Connections: Directly link layers from the contracting path to corresponding layers in the expanding path.
- Enable the network to combine low-level spatial details with high-level semantic features.

Fully Convolutional Architecture:

- Eliminates the need for fully connected layers.
- Allows the network to process images of arbitrary sizes efficiently.

Modified Expanding Path:

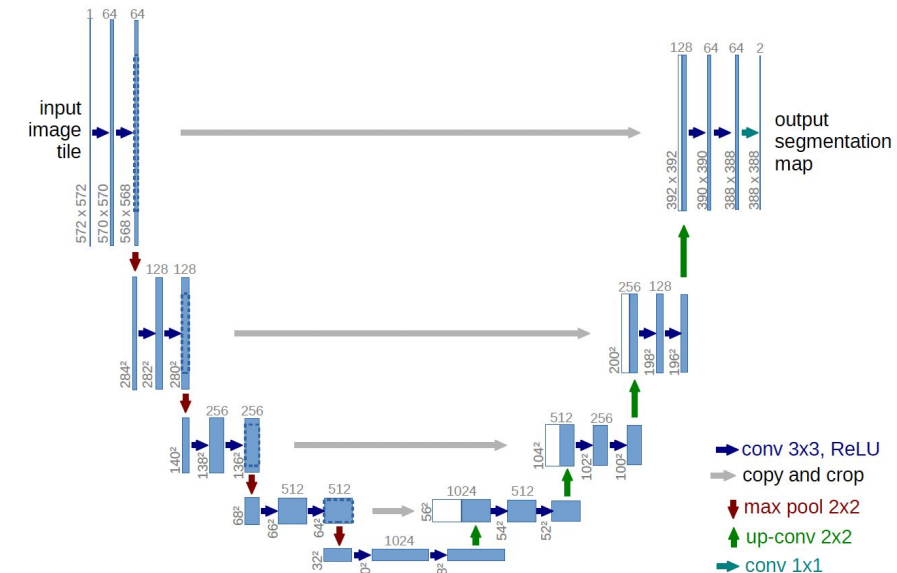
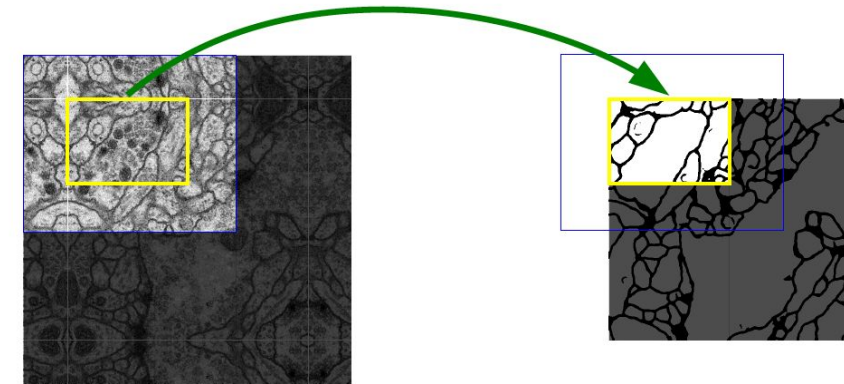
- Larger number of feature channels maintained throughout the expanding path.
- Ensures propagation of contextual information to higher-resolution layers.

Segmentation Without Borders:

- Uses only valid parts of convolutions to avoid artifacts at the edges.
- Allows for seamless segmentation of large images via the overlap-tile strategy:
 - Missing context at the image borders is extrapolated using mirroring.

Data Augmentation:

- Incorporates elastic deformations, making the network invariant to these transformations.



Training

Training Challenges:

- Limited amount of annotated training data, especially in biomedical tasks.
- Importance of handling imbalances between foreground and background classes.

Key Training Strategies:

- Loss Function:
 - Pixel-wise cross-entropy combined with a weighted loss.
 - Weights are assigned to balance class frequencies and emphasize separating boundaries between objects (e.g., touching cells).
- Data Augmentation:
 - Elastic deformations to simulate realistic variations (e.g., tissue stretching).
 - Shifts, rotations, and intensity variations to improve generalization.

Optimization:

- Stochastic Gradient Descent (SGD) with high momentum (0.99).
- Large input tiles used instead of large batch sizes to maximize GPU memory usage.

Resulting Benefits:

- Enhanced robustness to deformations and limited data availability.
- Improved boundary precision for comp

Mathematical expression for the loss function:

$$E = - \sum_{x \in \Omega} w(x) \cdot \log(p_{\ell(x)}(x))$$

$$p_k(x) = \frac{\exp(a_k(x))}{\sum_{k'=1}^K \exp(a_{k'}(x))}$$

where:

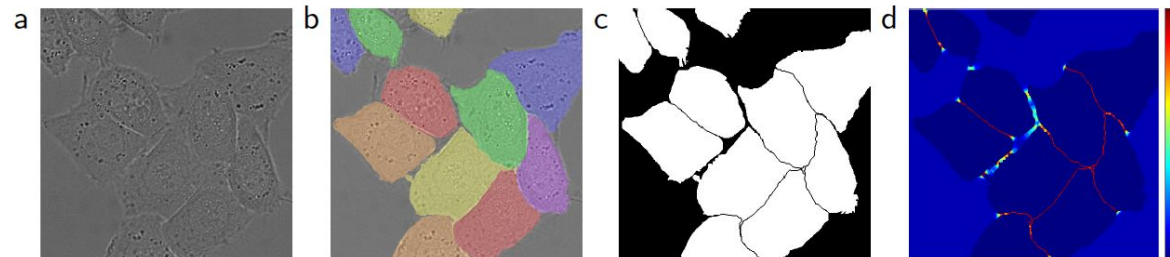
- Ω : set of all pixels in the image.
- $\ell(x)$: true class of pixel x .
- $p_{\ell(x)}(x)$: predicted probability for the true class $\ell(x)$.
- $w(x)$: weight map to handle class imbalance and emphasize object boundaries.

Weight Map:

$$w(x) = w_c(x) + w_0 \cdot \exp\left(-\frac{(d_1(x) + d_2(x))^2}{2\sigma^2}\right)$$

where:

- $w_c(x)$: weight for class balancing (e.g., foreground vs. background).
- $d_1(x)$: distance to the nearest object boundary.
- $d_2(x)$: distance to the second nearest boundary.
- w_0 : parameter controlling boundary importance (commonly $w_0 = 10$).
- σ : width of the boundary influence region (e.g., $\sigma = 5$).



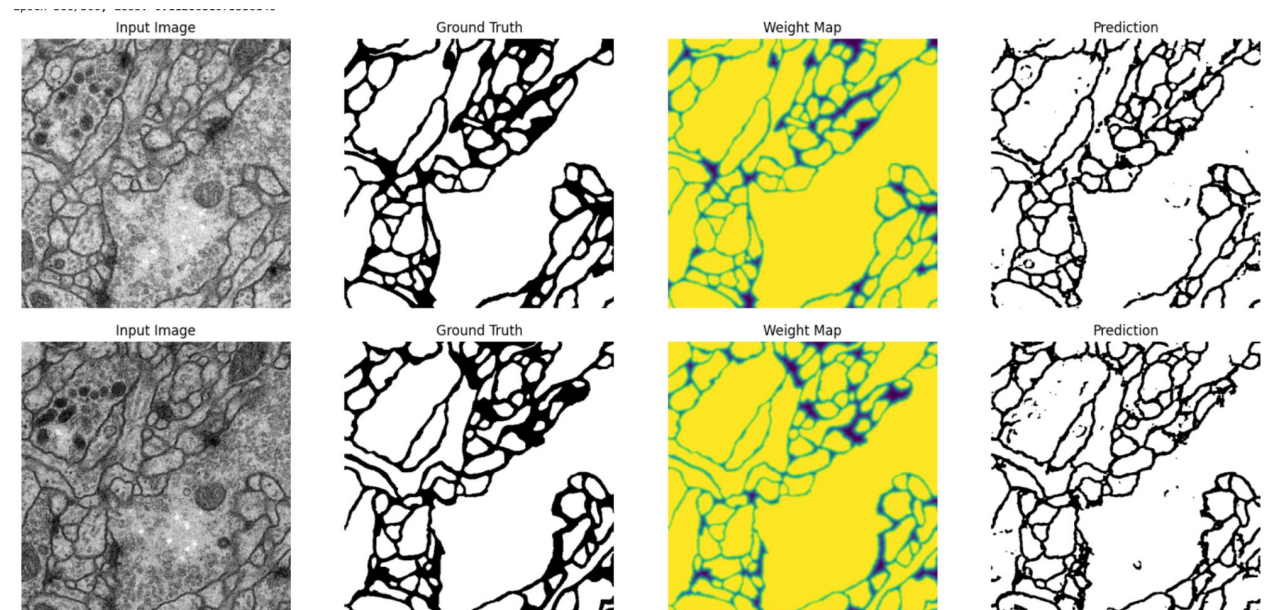
Results - Neuronal Segmentation (ISBI EM Challenge)

- Dataset and Task:
 - ISBI Challenge for Electron Microscopy (EM) segmentation.
 - Task: Segment neuronal structures from EM stacks (30 images).
- Performance Metrics:
 - Warping Error: 0.000353 (Best score in the competition).
 - Rand Error: 0.0382.
 - Significantly outperformed the sliding-window CNN approach (Warping Error = 0.000420).
- Impact:
 - Accurate segmentation even with a small training set (30 annotated images).
 - Demonstrates U-Net's ability to handle large, high-resolution images using the overlap-tile strategy.
 - Segmentation of a 512x512 pixel image takes less than 1 second on a modern GPU.

Rank	Group name	Warping Error	Rand Error	Pixel Error
	** human values **	0.000005	0.0021	0.0010
1.	u-net	0.000353	0.0382	0.0611
2.	DIVE-SCI	0.000355	0.0305	0.0584
3.	IDSIA [1]	0.000420	0.0504	0.0613
4.	DIVE	0.000430	0.0545	0.0582
	⋮			
10.	IDSIA-SCI	0.000653	0.0189	0.1027

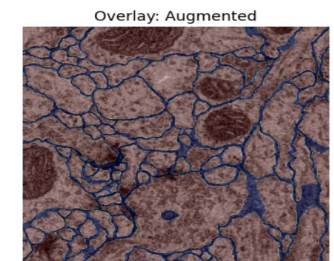
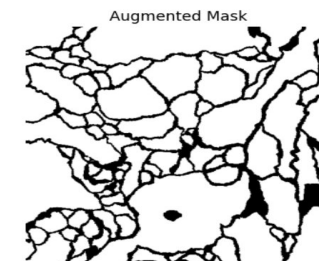
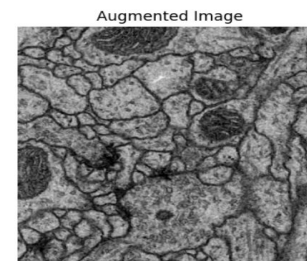
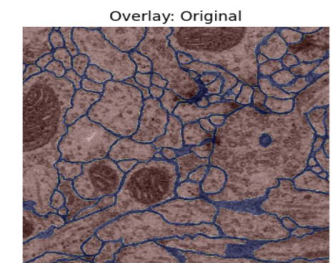
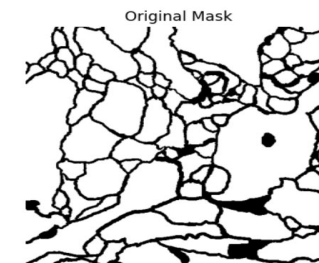
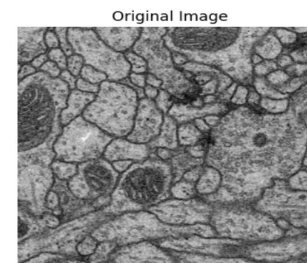
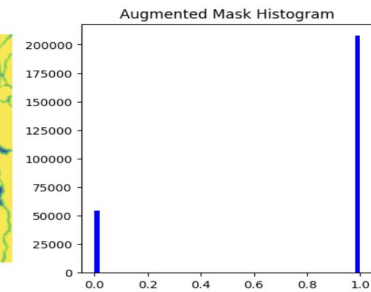
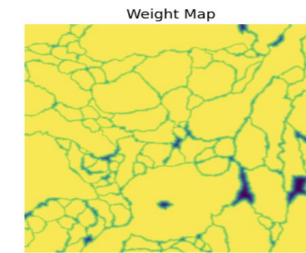
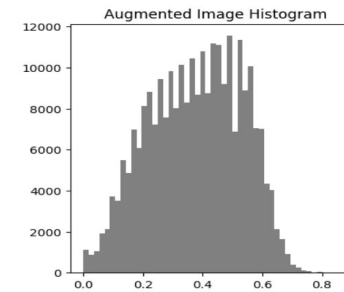
My Results - Neuronal Segmentation (ISBI EM Challenge)

- Weight Map:** The weight map implementation in your code is simpler than in the original paper. It provides insufficient emphasis on separating touching objects and balancing the background. This reduces model accuracy, particularly at object boundaries and when handling overlapping structures.
- Loss Function:** BCEWithLogitsLoss is used instead of cross-entropy with softmax. While suitable for binary tasks, it is less effective for multiclass segmentation. This may lead to a loss of accuracy when dealing with interrelated classes.
- Padding in Convolutions:** Padding is used to maintain spatial dimensions after convolutions, unlike the original U-Net. This simplifies model construction but may reduce spatial consistency in segmentation.
- Lack of Data Augmentation:** The paper emphasizes generating additional training data. If data augmentation in your code is minimal, the model is less likely to adapt to variations in the dataset.



My Results - Augmentation

```
def get_train_transform():
    p = 0.95
    return A.Compose([
        A.OneOf([
            A.HorizontalFlip(p=p),
            A.VerticalFlip(p=p),
            A.Transpose(p=p),
            A.RandomRotate90(p=p),
            A.ShiftScaleRotate(p=p, shift_limit=0.0625,
scale_limit=0.1, rotate_limit=45)
        ], p=1),
        A.GaussNoise(p=p, var_limit=(0, 20), mean=0,
per_channel=True),
        A.MultiplicativeNoise(p=p, multiplier=(0.9, 1.1),
elementwise=True),
        A.ElasticTransform(p=p, alpha=35, sigma=5,
approximate=True),
        A.RandomBrightnessContrast(p=p, brightness_limit=0.15,
contrast_limit=0.15),
        A.Normalize(mean=0.0, std=1.0),
        ToTensorV2()
    ])
```



Challenges

- **Insufficient Computational Resources:**
 - High memory requirements for training U-Net with weight maps and high-resolution images.
 - Limited batch size due to GPU constraints, which impacts training stability and results.
- **Complexity of Original Code:**
 - The original implementation in Caffe was difficult to interpret and replicate in PyTorch.
 - Specific details and custom features (e.g., weight map implementation) were not clearly explained, requiring additional assumptions.
- **Lack of Metric Descriptions:**
 - Evaluation metrics used in the original work were not explicitly stated.
 - No clear baseline to compare the performance of the replicated model.
- **Unavailable or Unclear Preprocessing Details:**
 - Key preprocessing steps (e.g., specific augmentations or normalizations) were not fully described.
 - Some assumptions about the dataset and transformations had to be made.
- **Ambiguity in Epochs and Training Configuration:**
 - The number of epochs, learning rate schedules, and other training hyperparameters were not specified in the original source.
 - Required experimentation to achieve reasonable results.
- **Unclear Metric for Weight Maps:**
 - The specific strategy for balancing weight maps, especially for overlapping objects, was not well-documented.
 - Required custom implementation based on inferred details, which may differ from the original.
- **Missing Reference Implementation Details:**
 - No available, runnable reference implementation to verify correctness.
 - Some critical details of the architecture (e.g., padding strategy) had to be inferred.
- **Incompatible or Unsupported Code (Java Script Example):**
 - Provided Java script was not directly runnable or convertible, complicating reproduction efforts.
 - Conversion from Java to Python required significant manual work without guaranteed correctness.